IT5034
Blockchain Technologies

Ethereum DApp for Banking

A Project Report

submitted by

Yuvaraj K	**2022506079**
Nithish S	**2022506099**
Krishnaa S	**2022506104**

Department of Information Technology
Madras Institute of Technology
Anna University, Chennai – 600 044

November 2025

Bonafide Certificate

Members:	Yuvaraj K	-	2022506079
	Nithish S	-	2022506099
	Krishnaa S	-	2022506104

Subject Code: IT5034

Subject Title: Blockchain Technologies

Certified that this project, titled **Ethereum DApp for Banking**, is the Bonafide work of **Yuvaraj K (2022506079)**, **Nithish S (2022606099)**, and **Krishnaa S (2022506104)**, who carried out the project work under the guidance of **Mrs. P. Seethalakshmi** during the period **July 2025 to November 2025.**

Submitted on: 30-11-2025

Table of Contents

Chapter	Title	Page No.
1	Introduction	1
2	Technology Stack	3
3	Workflow	7
4	Client-Side Application	13
5	Conclusion	17

1: Introduction

1.1: Overview

The rapid evolution of financial technologies has prompted a transformation in how banking services are delivered and accessed. Traditional centralized banking systems, while reliable, face limitations in transparency, trust, efficiency, and security. Blockchain technology presents a powerful alternative by enabling decentralized, transparent, and tamper-proof financial ecosystems. A blockchain-based decentralized banking application can empower users with direct control over their assets and transactions without the need for intermediaries, reducing operational costs and improving security and financial inclusion.

1.2: Background

Banking systems today rely heavily on centralized architectures, where banks and financial institutions control user data, validate transactions, and maintain ledgers. This centralization introduces single points of failure, susceptibility to fraud, and delays in transaction settlements. Blockchain, introduced with Bitcoin in 2009, is a distributed ledger technology that offers a way to manage transactions without a central authority. It ensures immutability, transparency, and consensus-driven validation.

With the rise of Decentralized Finance, blockchain is now being used to build decentralized applications or DApps that replicate and even enhance traditional financial services such as lending, payments, savings, and asset management. Leveraging smart contracts, these services operate autonomously and securely over blockchain networks, minimizing human intervention and institutional trust dependencies.

1.3: Problem Statement

Traditional banking systems are limited by centralization, which creates several issues:

- Users have limited visibility into how their data and funds are managed.
- Intermediaries and administrative overhead increase transaction costs.
- Centralized databases are prime targets for cyberattacks and data breaches.
- Many individuals in underserved regions lack access to basic services.

There is a growing need for an alternative system that is secure, cost-efficient, transparent, and accessible. A blockchain-based decentralized banking application addresses these concerns by removing intermediaries and empowering users with full control over their financial activities.

1.4: Objective

The objective of this project is to:

- Design and develop a decentralized banking application using blockchain.
- Ensure the system supports core banking functionalities such as account creation, peer-to-peer transfers, and the like via smart contracts.
- Enhance security, transparency, and user autonomy in transactions.
- Demonstrate the advantages of decentralization over traditional banking.
- Analyse performance, scalability, and compliance aspects of decentralized banking systems.

1.5: Scope

This report covers the following:

- Study and implementation of blockchain in the banking domain.
- Design of a decentralized architecture for banking using smart contracts.
- Implementation of basic financial services.
- Use of Ethereum or similar platforms for smart contract development.
- Evaluation of security, transaction costs, and user experience.
- Comparative analysis between centralized and decentralized models.

The scope does not include:

- Full regulatory compliance analysis.
- Real-world deployment with actual currencies (limited to test nets or token simulations).
- Integration with legacy banking infrastructure.

2: Technology Stack

2.1: HTML

Hypertext Markup Language is the standard markup language for documents designed to be displayed in a web browser. It defines the content and structure of web content. HTML markup consists of several key components, including those called tags (and their attributes), character-based data types, character references and entity references. HTML tags most commonly come in pairs like <h1> and </h1>, although some represent empty elements and so are unpaired, for example <img>. The first tag in such a pair is the start tag, and the second is the end tag (they are also called opening tags and closing tags).

2.2: CSS

Cascading Style Sheets (CSS) is a style sheet language used for specifying the presentation and styling of a document written in a markup language such as HTML or XML (including XML dialects such as SVG, MathML or XHTML). CSS is a cornerstone technology of the World Wide Web, alongside HTML and JavaScript. CSS is designed to enable the separation of content and presentation, including layout, colours, and fonts. This separation can improve content accessibility, since the content can be written without concern for its presentation; provide more flexibility and control in the specification of presentation characteristics; enable multiple web pages to share formatting by specifying the relevant CSS in a separate .css file, which reduces complexity and repetition in the structural content; and enable the .css file to be cached to improve the page load speed between the pages that share the file and its formatting.

2.3: JS

JavaScript, often abbreviated as JS, is a programming language and core technology of the World Wide Web, alongside HTML and CSS. Ninety-nine percent of websites use JavaScript on the client side for webpage behaviour. Web browsers have a dedicated JavaScript engine that executes the client code. These engines are also utilized in some servers and a variety of apps. The most popular runtime system for non-browser usage is Node.js.

JavaScript is a high-level, often just-in-time–compiled language that conforms to the ECMAScript standard. It has dynamic typing, prototype-based object-orientation, and first-class functions. It is multi-paradigm, supporting event-driven, functional, and imperative programming styles. It has application programming interfaces (APIs) for working with text, dates, regular expressions, standard data structures, and the Document Object Model (DOM).

2.4: jQuery

jQuery is a fast, small, and feature-rich JavaScript library. It makes things like HTML document traversal and manipulation, event handling, animation, and Ajax much simpler with an easy-to-use API that works across a multitude of browsers. With a combination of versatility and extensibility, jQuery has changed the way that millions of people write JavaScript.

2.5: Solidity

Solidity is an object-oriented, high-level language for implementing smart contracts. Smart contracts are programs that govern the behaviour of accounts within the Ethereum state. Solidity is a curly-bracket language designed to target the Ethereum Virtual Machine (EVM). It is influenced by C++, Python, and JavaScript. You can find more details about which languages Solidity has been inspired by in the language influences section. Solidity is statically typed, supports inheritance, libraries, and complex user-defined types, among other features. With Solidity, you can create contracts for uses such as voting, crowdfunding, blind auctions, and multi-signature wallets.

2.6: Node Package Manager

npm is the standard package manager for Node.js. In September 2022 over 2.1 million packages were reported being listed in the npm registry, making it the biggest single language code repository on Earth, and you can be sure there is a package for (almost!) everything. It started as a way to download and manage dependencies of Node.js packages, but it has since become a tool used also in frontend JavaScript. npm consists of three distinct components:

- the website
- the Command Line Interface (CLI)
- the registry

2.7: Truffle Framework

A world class development environment, testing framework and asset pipeline for blockchains using the Ethereum Virtual Machine (EVM), aiming to make life as a developer easier. With Truffle, we get:

- Built-in smart contract compilation, linking, deployment and binary management.
- Advanced debugging with breakpoints, variable analysis, and step functionality.
- Use console.log in your smart contracts
- Deployments and transactions through MetaMask with Truffle Dashboard to protect your mnemonic.

- External script runner that executes scripts within a Truffle environment.
- Interactive console for direct contract communication.
- Automated contract testing for rapid development.
- Scriptable, extensible deployment & migrations framework.
- Network management for deploying to any number of public & private networks.
- Package management with NPM, using the ERC190 standard.
- Configurable build pipeline with support for tight integration.

2.8: Ganache

Ganache is a personal blockchain for rapid Ethereum and Filecoin distributed application development. You can use Ganache across the entire development cycle; enabling you to develop, deploy, and test your DApps in a safe and deterministic environment. Ganache comes in two flavours: a UI and CLI. Ganache UI is a desktop application supporting Ethereum and Filecoin technology. Our more robust command-line tool, ganache, is available for Ethereum development. It offers:

- console.log in Solidity
- Zero-config Mainnet and testnet forking
- Fork any Ethereum network without waiting to sync
- Ethereum JSON-RPC support
- Snapshot/revert state
- Mine blocks instantly, on demand, or at an interval
- Fast-forward time
- Impersonate any account (no private keys required!)
- Listens for JSON-RPC 2.0 requests over HTTP/WebSockets
- Programmatic use in Node.js
- Pending Transactions

2.9: MetaMask

MetaMask is many things. For our purposes, it is a gateway to the Blockchain network. The MetaMask extension in a compatible browser such as Google Chrome or Brave or Microsoft Edge allows for web DApps to:

- Interact with users' Ethereum accounts.
- Send transactions and sign messages.
- Listen to account and network changes.

The MetaMask Wallet API is designed for web DApps that connect to the MetaMask browser extension. If you're building a desktop web-only DApp, the Wallet API provides everything you need. You can connect your DApp to users' MetaMask wallets by detecting MetaMask in their browsers and connecting to

their accounts. This approach allows you to detect multiple installed wallets and connect to them without conflicts.

MetaMask injects a global JavaScript API into websites visited by its users using the window.ethereum provider object. This API is specified by EIP-1193, and it allows DApps to request users' EVM accounts, read data from blockchains the user is connected to, suggest that the user sign messages and transactions, and more. The MetaMask Ethereum provider API contains the following:

- Properties - The provider contains a property that detects if a user has MetaMask installed.
- Methods - The provider contains methods that dapps can call. The request() provider method wraps the JSON-RPC API; dapps can use this provider method to call any RPC method.
- Events - The provider emits events that dapps can listen to.

2.10: Mocha Testing Framework

Mocha is a feature-rich JavaScript test framework running on Node.js and in the browser, making asynchronous testing simple and fun. Mocha tests run serially, allowing for flexible and accurate reporting, while mapping uncaught exceptions to the correct test cases.

3: Workflow

3.1: Project Structure

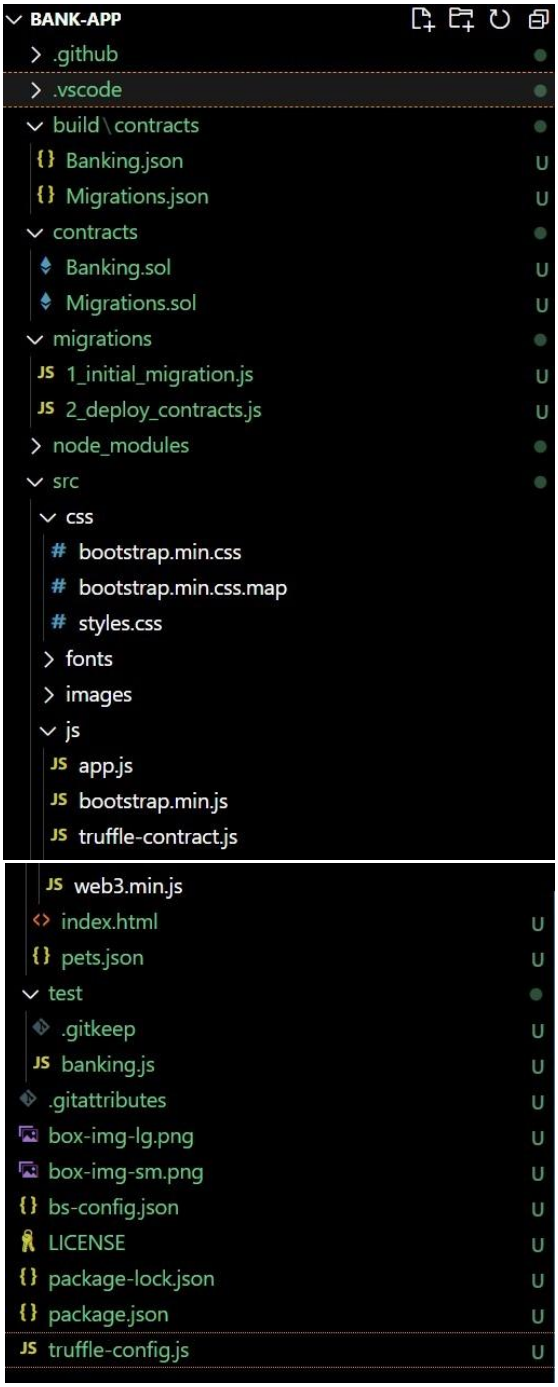

Fig 3.1: Project Structure

3.2: Architecture

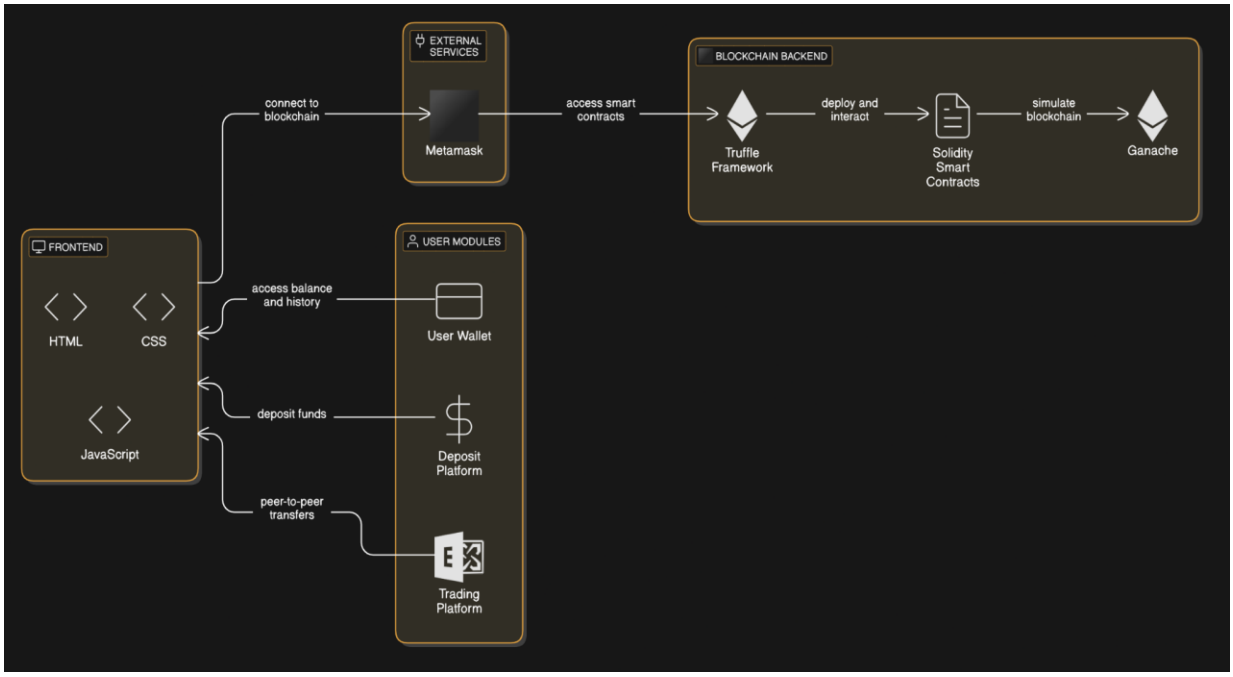

Fig 3.2: Architecture Diagram

The frontend is built using standard web technologies such as HTML, CSS, and JavaScript. Users can view their wallet balances, transaction history, deposit funds, and initiate peer-to-peer transfers directly from the browser. MetaMask serves as the primary external service, acting as a bridge between the frontend and the blockchain. MetaMask allows the frontend to access blockchain smart contracts and facilitates interactions with deployed contracts without exposing users' private keys or requiring them to leave the application interface. The User Wallet enables users to monitor their account balances and review transaction history. The Deposit Platform allows users to add funds to their DApp account. Meanwhile, the Trading Platform provides peer-to-peer transfer capabilities, enabling decentralized exchange of funds or tokens between users. The backend consists of the Truffle Framework, Solidity Smart Contracts, and Ganache.

This architecture allows users to securely interact with a decentralized application, enabling asset management and peer-to-peer transactions while leveraging the trust less and transparent nature of blockchain technology. The modular and layered design ensures scalability, ease of testing, and integration with blockchain services.

3.3: Flow of the Application

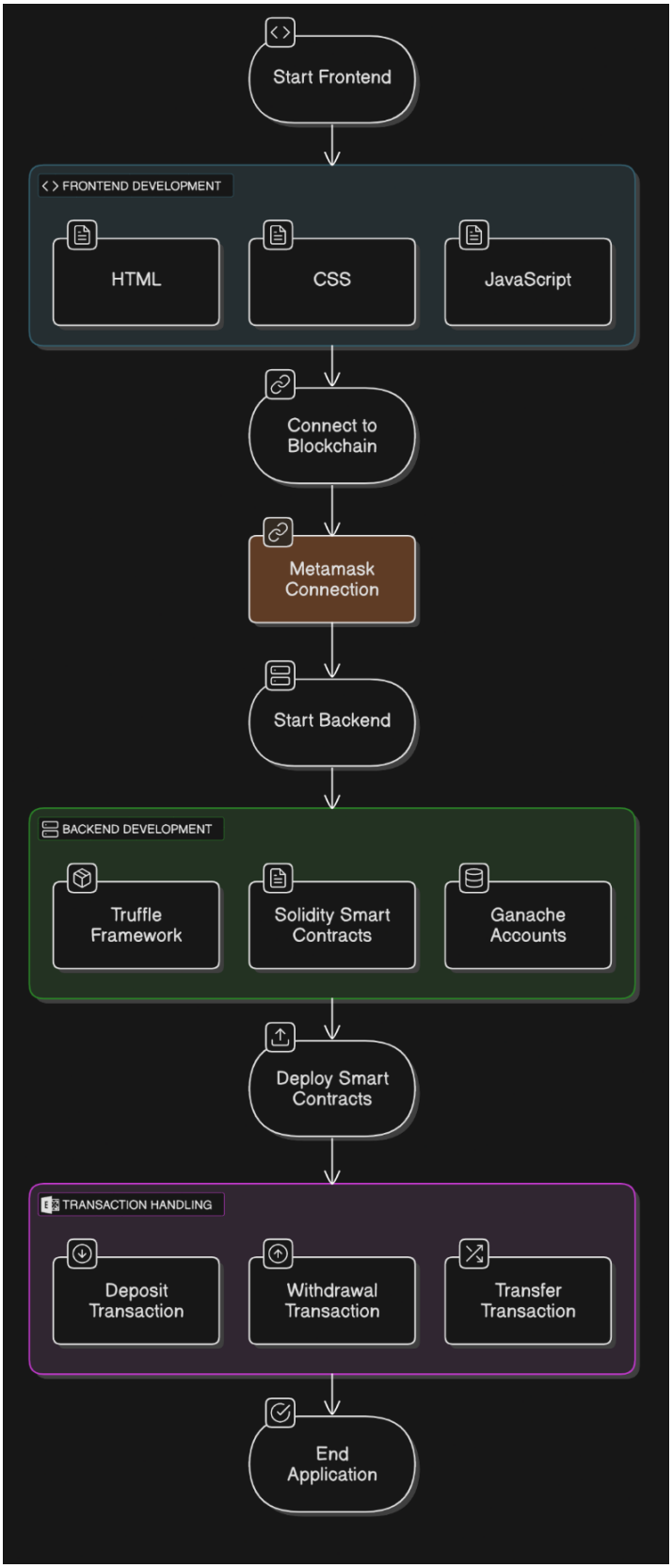

Fig 3.3: Flow Diagram

This flow diagram illustrates the lifecycle of a blockchain-based decentralized application (DApp) from frontend development to backend integration and transaction handling. The process begins with the initiation of the frontend, developed using core web technologies—HTML, CSS, and JavaScript—which together provide the user interface and client-side functionality of the DApp.

Once the frontend is ready, the next step is to connect the application to the blockchain. This is achieved through MetaMask, a browser extension that enables secure communication with the blockchain network. MetaMask handles user authentication and transaction signing, bridging the frontend and blockchain layers.

Following a successful connection, the backend development is initialized. This stage involves three key components: the Truffle Framework for compiling and deploying smart contracts, Solidity Smart Contracts that encapsulate the core logic and operations of the DApp, and Ganache Accounts, which simulate a local Ethereum blockchain environment for testing and development purposes.

After backend setup, the smart contracts are deployed onto the blockchain (simulated via Ganache in development). Once deployed, the application enters the transaction handling phase, where various user-initiated financial operations are managed. This includes deposit transactions (adding funds), withdrawal transactions (retrieving funds), and transfer transactions (peer-to-peer fund movement).

The process concludes with the termination of the application session, completing the full cycle of user interaction, blockchain connection, smart contract deployment, and transaction execution in the DApp.

3.4: Frontend Development and Initialization

The application lifecycle begins with the frontend development, where foundational web technologies—HTML, CSS, and JavaScript—are used to create an interactive user interface. The frontend facilitates user access to all core features of the DApp, including wallet balance views, deposit forms, transaction history, and peer-to-peer transfer inputs.

Once the UI is complete, the application proceeds to connect to the blockchain through MetaMask, a browser extension wallet. MetaMask plays a crucial role by acting as a key management system for the user's Ethereum accounts. It enables the DApp to:

- Request account access
- Retrieve wallet addresses
- Sign transactions securely
- Track account balances and activities

- Manage Ethereum test accounts

Through MetaMask, the DApp gains read and write access to the blockchain via the user's selected account and network.

3.5: Backend Development and Blockchain Simulation

After establishing the MetaMask connection, the backend development process is initiated. This layer involves the use of several Ethereum development tools:

- Truffle Framework: Serves as the project scaffolding and smart contract management tool. It compiles, migrates (deploys), and interacts with smart contracts in a structured environment.
- Solidity Smart Contracts: The core application logic resides in these contracts, which handle deposits, withdrawals, transfers, and balance management. Functions are written to include access control, value validations, and event emission for off-chain tracking.
- Ganache: A local in-memory Ethereum blockchain used for testing and development. Ganache provides a set of deterministic Ethereum accounts preloaded with test ETH, allowing developers to simulate blockchain interactions without incurring real gas costs or relying on testnets.

Developers launch the backend environment using Truffle, and smart contracts are deployed onto the local Ganache blockchain. Contract migration scripts manage the deployment process, ensuring all contracts are compiled and deployed in the correct sequence.

3.6: Smart Contract Deployment and Interaction

Once development is complete, the application enters the deployment phase. Using Truffle, the compiled Solidity contracts are deployed to Ganache via migration scripts. These contracts include functionality for:

- Storing and managing user balances
- Validating deposits and withdrawals
- Facilitating peer-to-peer transfers
- Emitting blockchain events (e.g., DepositMade, FundsTransferred, WithdrawalCompleted) for frontend tracking

Upon deployment, each contract is assigned a unique Ethereum address, which the frontend uses (via Web3.js or Ethers.js) to interact with the blockchain.

3.7: Automated Testing with Mocha

To ensure contract integrity, automated testing is conducted using Mocha, a JavaScript-based testing framework integrated within the Truffle suite. A total of 8 test cases were written and executed, covering key functional areas:

- Deployment validation

- Account balance initialization
- Successful deposits and event emissions
- Withdrawal operations with balance checks
- Peer-to-peer transfer between test accounts
- Access control verification (e.g., preventing unauthorized withdrawals)
- Edge case handling (e.g., zero-value transfers)
- Reversion of invalid transactions with proper error messages

All 8 test cases passed, confirming that the smart contracts behave as expected under normal and boundary conditions. The tests used the Ganache-supplied test accounts and relied on Web3 calls to simulate real transaction scenarios.

3.8: Transaction Handling

With smart contracts successfully deployed and tested, the DApp transitions to the transaction execution phase. Users are able to perform the following operations through the frontend:

- Deposit: Initiated via MetaMask, the transaction sends ETH to the contract address and updates the sender's balance in the smart contract state.
- Withdraw: Users can withdraw available funds back to their MetaMask wallet. The contract validates the balance and processes the transfer.
- Transfer: Enables peer-to-peer fund transfers between registered accounts. The contract adjusts balances and emits events for tracking.

Each transaction triggers corresponding smart contract functions and requires user approval through MetaMask. After submission, the transaction is mined by Ganache and the result is reflected in the DApp interface.

3.9: Completion and Application Shutdown

After transactions are completed, the DApp session can be safely terminated. All blockchain interactions are recorded in the local ledger simulated by Ganache. Since data on Ganache is not persistent between sessions by default, we may export or snapshot the chain state for further testing or auditing.

4: Client-Side Application

4.4: Frontend Client

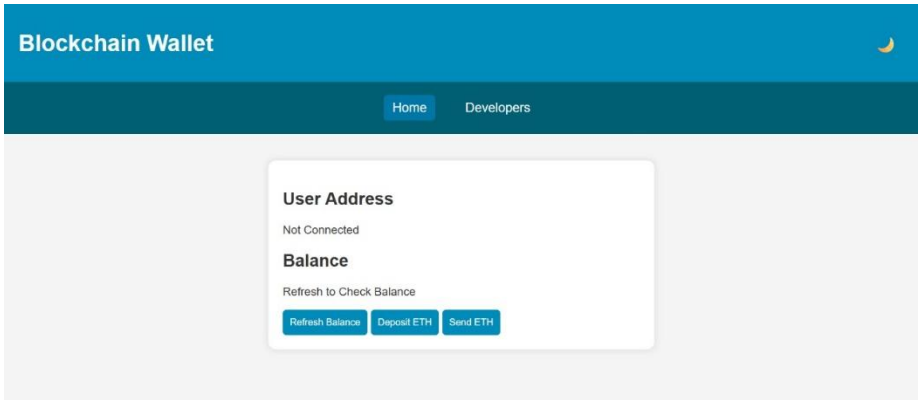

Fig 4.1: Home Page – No User Logged In

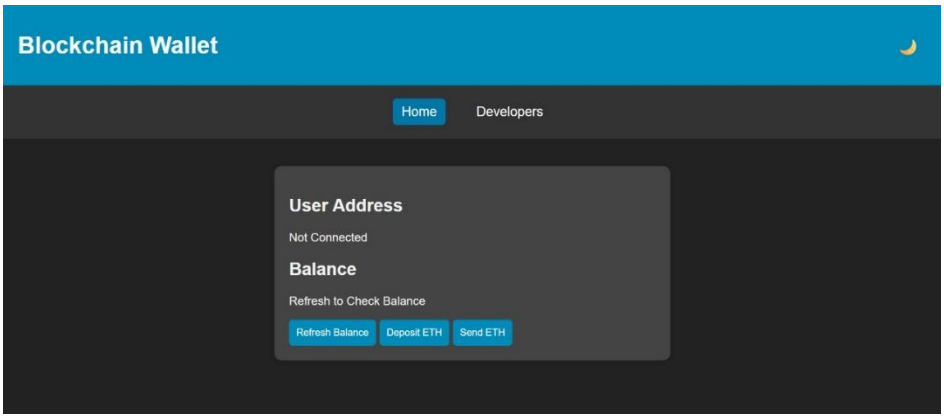

Fig 4.2: Dark Mode

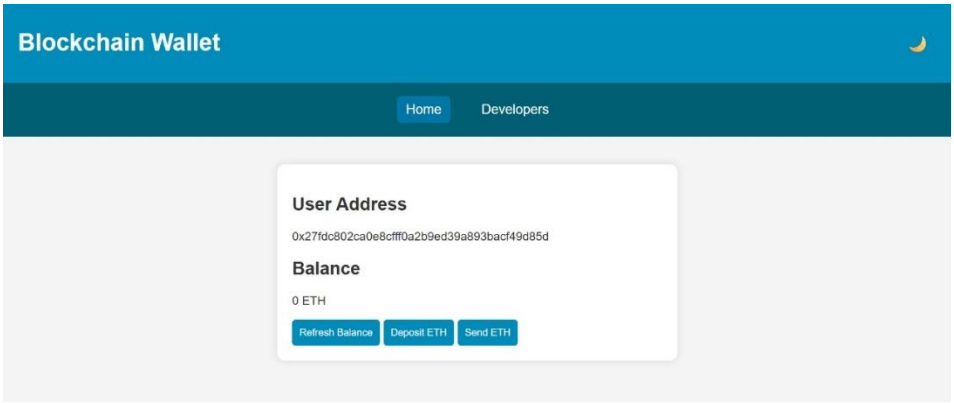

Fig 4.3: After Connecting

Fig 4.4: Deposit

4.5: Ganache Simulation

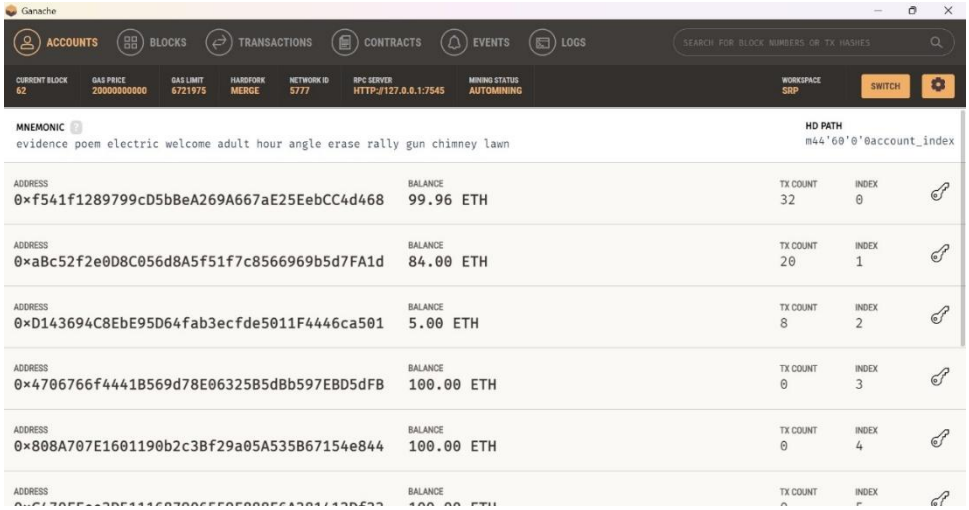

Fig 4.5: Ganache Simulated Accounts

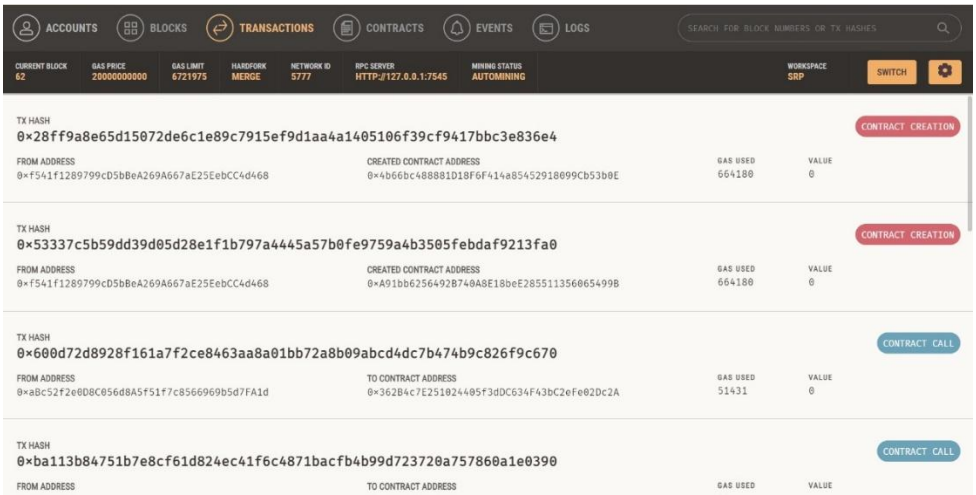

Fig 4.6: Transaction History in Ganache

4.6: MetaMask Connection

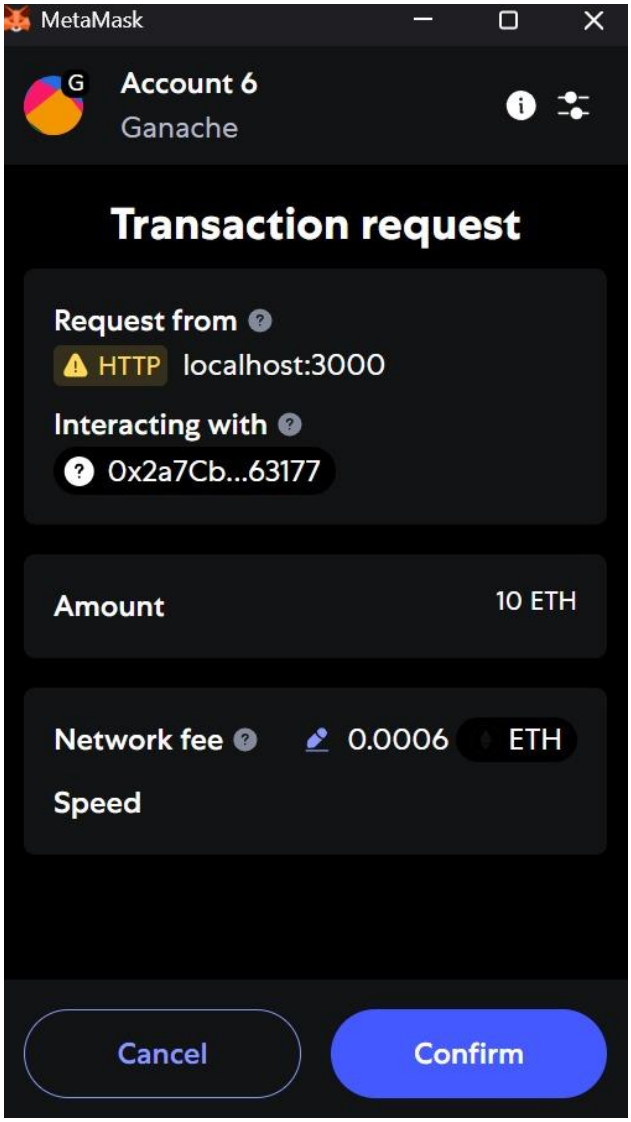

Fig 4.7: Transaction Request

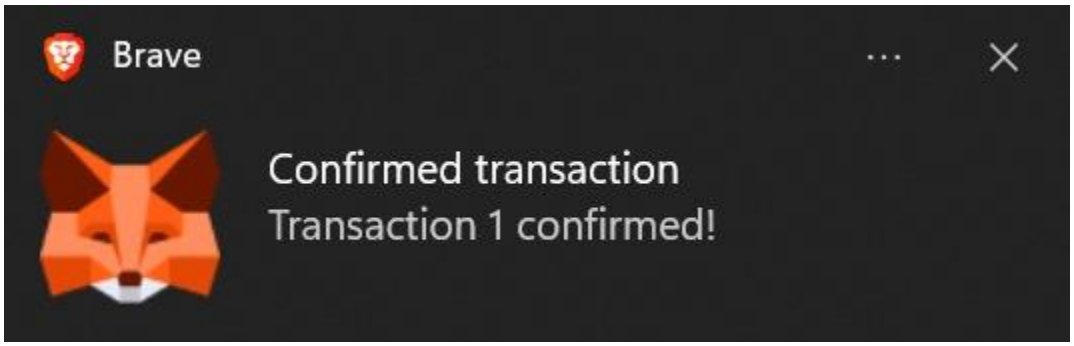

Fig 4.8: Transaction Confirmation

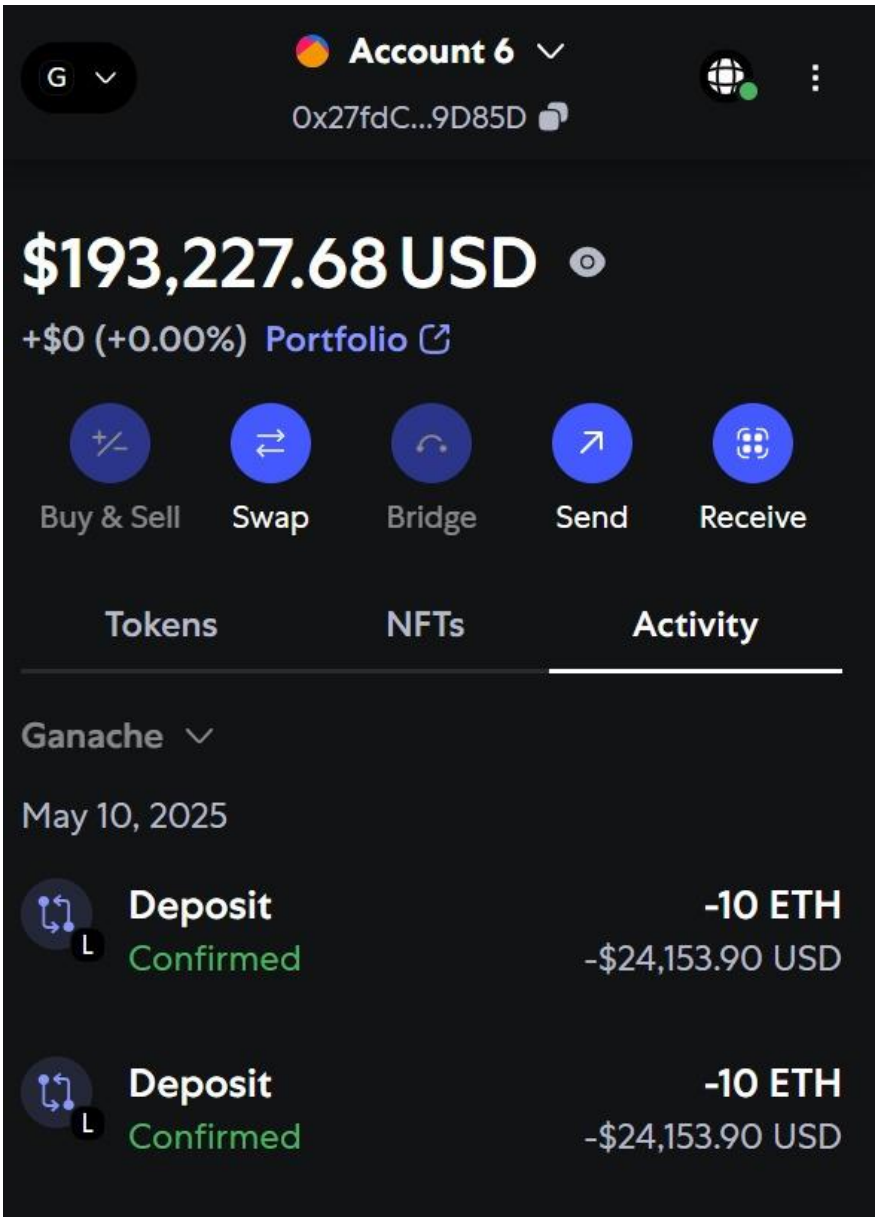

Fig 4.9: Local Transaction History

5: Conclusion

The proposed blockchain-based decentralized banking application represents a significant step toward redefining traditional financial systems by leveraging the transparency, security, and autonomy offered by blockchain technology. The current architecture provides a functional model that integrates key components such as smart contracts, wallet interfaces, and blockchain simulation environments.

However, to transition from a proof-of-concept to a scalable, secure, and user-friendly financial platform, several enhancements are necessary. Through continuous refinement and adherence to best practices, this decentralized banking system can evolve into a robust, secure, and inclusive financial solution for users worldwide. Potential improvements include:

- Backend Middleware Integration:
- Enhanced Authentication and Identity Management:
- Scalability through Layer-2 Solutions
- Off-Chain Data Management
- Modular Smart Contract Design
- Frontend Modernization
- Smart Contract Testing and Security
- Multi-Chain and Interoperability Support